560 lab5 README

Team Member (Group 10)

- **Mingtao Ding** | USC ID: 5538797503
- **Ke Wu** | USC ID: 3385282229
- **Yi-Hsien Lou** | USC ID: 1210391283

Github Link: https://github.com/ke101/labs_560_MKB/tree/main/lab5

Environment Setup

1. Create a `.env` file under the lab folder to configure MySQL connection parameters.
2. Create a Python virtual environment: `python -m venv venv`.
3. Activate the virtual environment and install packages from `requirements.txt`.

Running the Pipeline

Stage	Command	Purpose
Scrape	`python scrape [data_number] --method [bs4/api]`	Collects raw Reddit data.
Preprocessing & Vectorize	`python preprocessing.py vectorize.py`	Cleans and enriches the raw data. Embedding
Cluster & Keywords	`python cluster/keywords [mysql username and password]`	Runs clustering and keyword extraction.
Total Pipeline	`python main.py [interval] [other parameters]`	Executes the full pipeline.

Database Schema

1. `raw_posts` (The Source Data)

- **Purpose:** Stores raw data scraped directly from the Reddit API (or PRAW).
- **Primary Keys:** Internal auto-incrementing `id` and unique Reddit `post_id`.
- **Content:** `title`, `selftext` (body), `subreddit`, and `author` details.
- **Metrics:** `score`, `upvote_ratio`, `num_comments`.
- **Timestamps:** Original Reddit creation time (`created_utc`) and scrape time (`scraped_at`).

2. `cleaned_posts` (The Processed Data)

- **Purpose:** Holds sanitized and enriched posts, prepped for machine learning.
- **Relationship:** 1-to-1 relationship with `raw_posts` (updates/deletes cascade).
- **Anonymization:** Replaces raw author name with an anonymized hash (`author_anon`).
- **Cleaned Content:** Processed versions of `title_clean` and `body_clean`.
- **Enrichment:** `ocr_text`, comma-separated `keywords`, and NLP `topic` labels.
- **Formatting:** Standard SQL DATETIME format for creation time (`created_dt`) and a promotional content flag (`is_ad`).

3. `post_vectors` (The Embeddings)

- **Purpose:** Stores mathematical representations (vectors) of the text.
- **Vector Storage:** The embeddings are stored as a serialized NumPy array (`vector`).
- **Model Metadata:** Vector dimensionality (e.g., 100) and model name (defaulting to 'doc2vec').

4. `cluster_results` (The K-Means Output)

- **Purpose:** Maps individual posts to generated clusters.
- **Cluster Mapping:** Associates `post_id` with a specific `cluster_id` and execution `run_id`.
- **Geometry:** Records `distance_to_center` and flags the most representative post (`is_nearest`).

5. `cluster_keywords` (The Cluster Keywords)

- **Features:** Links a `cluster_id` and `run_id` to a specific, defining *keyword*.

Workflow Breakdown

1. `scrape.py`

- **Role:** Primary Reddit web scraper for data collection.

- **Core Methods:**
  - **PRAW (API):** Supports the official Reddit API (complete data, stable rate limiting, but requires a key).
  - **BS4 (Web Parsing): Actively used method** to parse `old.reddit.com` directly, bypassing the need for an API key.
- **Key Features:**
  - Multi-Subreddit Support & Global Deduplication.
  - Three-tier Ad Detection Strategy.
  - Optional Image Downloading.
  - Resilience with automatic retry using exponential backoff.
  - Optimized Database I/O with batch insertions (200 posts/batch).
  - **Idempotency:** Uses `ON DUPLICATE KEY UPDATE` to safely rerun and update existing post metrics.

2. `database.py`

- **Role:** Manages the database connection and interface.
- **Core Architecture:**
  - **Connection Pooling:** Maintains a pool of 5 connections for performance.
  - **Context Managers:** Safely handles commits and rollbacks for database transactions.
  - **Singleton Pattern:** Ensures only one `DatabaseManager` instance is active.
- **Key Interfaces:**
  - **Data Collection:** `insert_raw_post()`, `insert_raw_posts_batch()`.
  - **Preprocessing:** `get_unprocessed_posts()`, `insert_cleaned_post()`, `insert_vector()`.
  - **Clustering:** `get_all_vectors()`, `save_cluster_results()`, `save_cluster_keywords()`, `get_cluster_posts()`, `search_clusters_by_keyword()`.

3. `preprocess.py`

- **Role:** Preprocesses raw content.
- **Steps:**
  - **Text Normalization:** Strips HTML, removes URLs and non-alphanumeric characters, converts to lowercase.
  - **Anonymization & Formatting:** Hashes the author's username (`author_anon`) and converts the Unix timestamp to SQL datetime (`created_dt`).
  - **Topic Inference:** Scans for keywords related to cybersecurity topics (e.g., "ransomware") to assign a `topic` label.
  - **Keyword Extraction:** Extracts the top 10 most common words (longer than 4 characters) as a comma-separated string (`keywords`).
  - **Batch Processing:** Continuously processes batches of raw posts until none remain.

4. `vectorize.py`

- **Role:** Converts cleaned text into numerical vectors (embeddings).
- **Steps:**
    - **Identify Missing Vectors:** Finds `post_id`s in `cleaned_posts` without a matching vector in `post_vectors`.
    - **Model Training:** Creates and trains a **Doc2Vec** model (100-dimensional) using the `gensim` library on the entire cleaned corpus.
    - **Generate & Save:** Infers the 100-dimensional vector for missing posts and saves them in batches to the `post_vectors` table.

5. `cluster.py`

- **Role:** Groups posts into categories using K-Means.
- **Steps:**
    - **Fetching Data:** Reads all data from `post_vectors` into a Pandas DataFrame.
    - **Decoding Vectors:** Converts stored vector bytes back into NumPy numeric arrays.
    - **Clustering:** Applies the **K-Means algorithm** from `scikit-learn` to group vectors into exactly **5 clusters** (`n_clusters=5`).
    - **Saving Results:** Appends the `post_id` and the generated `cluster_id` mapping to the `cluster_results` table.

6. `keyword.py`

- **Role:** Analyzes clusters to identify common words and define themes.
- **Steps:**
    - **Fetching Data:** Performs a SQL `JOIN` on `cluster_results` and `cleaned_posts` to get `post_id`, `cluster_id`, and `title_clean`.
    - **Text Preprocessing:** Runs `preprocess_text` to lowercase, strip non-alphabetic characters, and remove common English stop words (`nltk`).
    - **Extracting Keywords:** Uses a `Counter` to find the **top 10 most common words** for each unique cluster.
    - **Formatting and Saving:** Concatenates the top 10 words and appends the final list of keywords into the `cluster_keywords` table.

7. `main.py`

- **Role:** Automate the whole process.
- **Steps:**
    - Run `scrape.py` at a user-input time interval.
    - Preprocess the data and update the database accordingly.
    - After stopping scraping, find the closest cluster based on the distance to the centroid and label the new data. (Future Work: If it is not close to any cluster based on some criteria, the whole clustering process should be rerun).